

Water Chlorination ICS Security

Team Project Briefing & Roadmap

Read-only analysis · Source of truth: repository at `HEAD = origin/main = fc79a4a`

Working tree clean · 286 tests passing

No code, tests, or configuration were changed to produce this document.

- IMPLEMENTED (in code, tested) · ● PLANNED (agreed, not built)
● DEFERRED (deliberately postponed, with a reason) · ○ PROPOSED FUTURE (idea, not yet decided)

1. Executive Project Summary

What we are solving

Industrial control systems (ICS) that run physical processes — here, a **water treatment / chlorination plant** — are increasingly targeted by cyber-attacks that manipulate sensors and actuators. Traditional IT security (firewalls, signatures) does not see a *process* being pushed into an unsafe state. We built a **process-aware anomaly detection platform**: it watches the plant's live sensor/actuator telemetry, learns what "normal operation" looks like, and raises an operator alert when the physical process deviates — even when no classic IT signature fires.

Why this is an OT/ICS security problem

The threat is not a malformed packet; it is a **physically plausible but wrong process state** (e.g. a pump commanded on when it should be off, a chemical dosing analyzer drifting). Detecting that requires modelling the *joint behaviour of 45 process variables over time*, which is an OT problem, not an IT one.

Why water chlorination matters

Chlorination is a safety-critical step: too little → unsafe water, too much → toxic. An attacker who quietly manipulates dosing or the sensors reporting it can cause real physical harm while telemetry "looks fine" field-by-field. This is exactly the kind of subtle, multivariable manipulation our detector targets.

Roles of each piece

- **SWaT dataset** — a real, labelled testbed dataset (Secure Water Treatment, iTrust/SUTD) of a 6-stage water treatment plant under normal operation and 36 launched attacks. It is our stand-in for live plant telemetry.
- **TranAD** — the anomaly-detection model (a transformer-based reconstruction model). It is the **selected, final MVP detection engine** — model selection is closed.
- **Replay** — streams the historical SWaT data through the system in order, as if it were live telemetry, so we can demonstrate detection end-to-end.
- **Alert engine** — turns a stream of per-window "anomaly / normal" decisions into **incidents**: one continuous anomaly = one alert with a lifecycle (open → close), deduplicated, with severity and contributing sensors.
- **Dashboard** — an operator view: system health, live detection status, active alert, alert history.

What makes this more than "just an ML model"

The ML is one stateless component. Around it we built a **real system**: a data-provenance-verified preprocessing contract, a stateless HTTP inference API, a stateful incident/alerting layer, a monitoring API, and an operator dashboard — plus 286 tests that protect the contracts. The project is fundamentally a **software + OT-security engineering** effort that productionizes a validated model, not a modelling exercise.

A. 30-second explanation (for a supervisor)

"We protect a water chlorination plant from cyber-attacks by watching its live process data. A transformer anomaly-detection model called TranAD learns normal plant behaviour from the SWaT dataset and flags abnormal process states. Flagged windows become deduplicated operator alerts shown on a dashboard, with the exact sensors that drove each alert. We have a working end-to-end MVP; the remaining work is deployment, persistence, and DevSecOps."

B. 2-minute explanation (for a teammate)

"The plant produces 45 process readings per second. We take a 30-sample rolling window (about 150 seconds of history) and feed it as a 30×45 matrix into TranAD, which tries to reconstruct the latest reading. If reconstruction error exceeds a calibrated threshold, that window is anomalous. A `RollingWindow` component owns the buffer; a stateless FastAPI `/score` endpoint runs the model; an `AlertEngine` groups consecutive anomalous windows into a single incident that opens, tracks its peak severity and top contributing sensors, and closes when the plant returns to normal. A monitoring API and a zero-build dashboard

expose all of this. We replay real SWaT data (normal and attack segments) to demonstrate it. Everything is covered by 286 tests. We deliberately kept the model, detector, buffering, and alerting as clean separate layers so we can later containerize and deploy to the cloud."

C. 5-minute technical explanation (for an examiner)

"The pipeline is: SWaT clean-Attack_v0 CSVs → **SWaTReplay** (ordered telemetry) → **RollingWindow** (owns the 30-sample stride-1 buffer, emits raw 30×45 windows) → HTTP **POST /score** (thin, stateless FastAPI boundary that validates shape and feature order) → **TranADDetector** (stateless: StandardScaler → float32 → C-order flatten to 1350 → MinMaxScaler clip=False → reshape → TranAD reconstruction of the last timestep → score) → **AlertEngine** (the one stateful stage: dedup + open/close incident lifecycle, heuristic severity, verbatim feature attribution) → monitoring endpoints **/status**, **/alerts**, **/alerts/active** → a polling dashboard. TranAD was selected after a research comparison of multiple models; its five artifacts are treated as immutable. We reverse-engineered and documented the exact preprocessing contract because the export script was lost and the notebooks had no saved outputs. We found real data-quality defects in SWaT (a duplicated normal block that froze 6 of 45 features and contaminated threshold calibration) and reconstructed a clean replay dataset. We report the honest **point-wise F1 = 0.7822** at the shipped threshold with a documented ~3% false-positive-rate caveat — not the inflated point-adjusted or test-searched F1 figures. Threshold recalibration is deliberately deferred. What remains is persistence, containerization, CI/CD with security gates, and cloud deployment; those directory scaffolds exist but are empty."

2. Complete System Architecture

```
SWaT clean Attack_v0 CSVs  (local, licensed, gitignored)
    |
    v
SWaTReplay                  simulator/swat_replay.py  [IMPLEMENTED]
    | ordered TelemetryRecord stream (normal/attack segments, paced)
    v
RollingWindow               simulator/window_buffer.py [IMPLEMENTED]  <- OWNS the 30-sample buffer
    | raw 30x45 Window (stride 1; nothing until buffer full)
    | window_to_request() (pure serialization)  simulator/pipeline.py [IMPLEMENTED]
    v
POST /score                 backend/app.py + schemas.py [IMPLEMENTED]  <- THIN, STATELESS boundary
    | validate shape/finiteness/feature-order -> delegate
    v
TranADDetector              ml/src/detector.py [IMPLEMENTED]          <- STATELESS
    (preprocessing.py -> model.py -> scoring.py)
    | DetectionResult {anomaly_score, threshold, is_anomaly, 45 feature_errors}
    v (best-effort, swallowed -- never breaks /score)
AlertEngine                 alerting/engine.py + alert.py [IMPLEMENTED] <- ONLY stateful stage; IN-MEMORY
    | Alert (OPEN -> dedup -> CLOSE), severity heuristic, verbatim attribution
    v
/status /alerts /alerts/active backend/app.py [IMPLEMENTED]          <- read-only monitoring over app.state
    | poll (default 2s), CORS allow-list (not wildcard)
    v
Operator Dashboard          frontend/index.html [IMPLEMENTED]         <- vanilla HTML/JS, zero-build
```

Component-by-component

Component	Responsibility	Input	Output	Tech	State	Key dependencies	Why it exists
SWaTReplay	Stream clean Attack_v0 as ordered telemetry, expose normal/attack segments, pace playback	Clean Attack_v0 series (via <code>ml.src.dataset.load_attack_v0</code>)	<code>TelemetryRecord</code> stream	Python, numpy	Stateless iterator (delegates data load)	<code>ml.src.dataset</code>	Turn a static CSV into a realistic live-telemetry source
RollingWindow	Own the 30-sample buffer ; emit complete 30×45 raw windows, stride 1	One telemetry record per <code>push()</code>	<code>Window(values 30×45, timestamps, feature_names, labels)</code> or <code>None</code> during warm-up	Python <code>deque</code> , numpy	Stateful (the raw buffer)	none (duck-typed input)	Keep the detector/API stateless; single place that knows "last 30 samples"
pipeline.py	Producer/client glue: serialize window → call injected <code>Scorer</code> → shape <code>EndToEndResult</code>	<code>Window</code> + injected <code>Scorer</code>	<code>EndToEndResult</code>	Python, httpx (lazy)	Stateless	<code>simulator</code> only (NOT torch/fastapi/ml.src)	Wire layers over HTTP without coupling them
FastAPI backend	Thin boundary: validate → delegate → shape; host monitoring endpoints	<code>/score</code> body (30×45 + names); GET monitoring	JSON responses	FastAPI, Pydantic	Holds detector + AlertEngine on <code>app.state</code> ; no buffering/streaming state	<code>ml.src.detector</code> , <code>alerting</code>	Network boundary; enforce the feature contract
TranADDetector	Preprocess + run TranAD + threshold decision	Raw 30×45 window	<code>DetectionResult</code> (+ 45 per-feature errors)	PyTorch, scikit-learn, joblib	Stateless	artifacts (read-only)	The detection engine
AlertEngine	Group consecutive anomalies into deduplicated lifecycle-managed incidents	Detector decisions (duck-typed)	<code>Alert</code> (OPEN→CLOSED)	Pure Python	Stateful, in-memory only	none (duck-typed)	Only place that knows 50 anomalous windows = 1 incident
Dashboard	Operator view via polling	API JSON	Rendered HTML	Vanilla HTML/CSS/JS	Holds last render only	backend API	Human-facing monitoring, zero build tooling

Explicit answers to the key questions

- **Who owns the rolling buffer?** `simulator.window_buffer.RollingWindow` — nobody else.
- **Is TranADDetector stateful or stateless? Stateless.** One 30×45 window in → one decision out. No memory between calls.
- **What does FastAPI do?** Validates request shape/finiteness/duplicate-names (422), enforces feature *identity and order* against `feature_names.json`, delegates to the detector, shapes the response, and hosts the read-only monitoring endpoints. It holds the shared detector and the in-memory AlertEngine on `app.state`.
- **What does FastAPI NOT do?** It does **not** buffer telemetry, does **not** keep streaming state, does **not** re-implement preprocessing/scoring, and does **not** re-implement dedup/lifecycle/severity. Monitoring updates are best-effort and swallowed so they can never break `/score`.
- **What does the AlertEngine do?** Converts per-window decisions into incidents: opens on first anomaly, deduplicates/extends across consecutive anomalous windows (tracking peak score, peak attribution, severity, window count, span), and closes on return to normal. Deterministic alert ids.
- **Where is alert state stored? In memory only**, on `app.state.alert_engine`. Lost on backend restart. No database.
- **How does the dashboard talk to the backend?** HTTP **polling** (default 2 s) of `/status`, `/alerts/active`, `/alerts`; cross-origin allowed via an explicit dev CORS allow-list (not a wildcard). Configurable via `?api=` and `?poll=` query params.
- **How does the system identify an attack?** TranAD reconstructs the last timestep of the window; the reconstruction error is the anomaly score; `is_anomaly = score > threshold` (strictly greater).
- **How does a detected anomaly become an alert?** `/score` produces a `DetectionResult`; the backend feeds it to `AlertEngine.process(...)`; the engine opens a new `Alert` (or extends the open one), copying the detector's top contributing features verbatim and computing a heuristic severity.

3. ML / TranAD Explanation

Models investigated → why TranAD

Multiple anomaly-detection models were trained and compared during the research phase (in `research/notebooks/` — SWaT notebooks plus a WADI experiment). After comparison, **TranAD was selected** as the MVP detector. **Model selection is CLOSED.** ⚠ *Do not propose replacing TranAD, and do not retrain — the artifacts are immutable.* (The research comparison tables were never committed to git; they lived in meeting slides — a known documentation gap.)

Why TranAD is the actual MVP model

It is declared so in `PROJECT_CONTEXT.md`, and the entire production stack (`ml/src`, artifacts, golden vectors, tests) is built around its exact contract. The five artifacts in `ml/artifacts/swat_TranAD/` are the source of truth.

Architecture used (recovered from tensor shapes, documented in provenance §4)

`TranAD(feats=45, window=30)`, `d_model = 90` ($= 2 \times \text{features}$), `nhead = 45`, `dim_feedforward = 16`, `dropout = 0.1`, 1 transformer-encoder layer, 2 decoders sharing one `fcn`. `model.pt` is a `state_dict` of 51 tensors (~180,993 params). The `TranAD`/`PositionalEncoding` classes were copied **verbatim** from the notebook because `state_dict` keys bind to exact attribute names.

Input / preprocessing contract (`ml/src/preprocessing.py`, provenance §3)

- **Input dimensions:** 30 timesteps $\times$ 45 features (a 30 $\times$ 45 matrix per decision).
- **Window size:** 30. **Stride:** 1. **Subsample:** 5 (`::5`) → effective **5-second sample period**.
- **Feature count:** 45 (6 near-constant columns dropped from SWaT's 51 process columns: P202, P401, P404, P502, P601, P603).
- **Chain:** `StandardScaler.transform` → cast to **float32** → **C-order flatten** to 1350 (`index = t*45 + f`) → `MinMaxScaler.transform` (`clip = False`) → reshape to (30, 45).
- **Three silently-failing properties, each regression-tested:** C-order flatten, clip=False, 1350 columns.
- **Production divergence:** rejects **non-finite** input (no `ffill/fillna`) — a stuck/absent channel is an operational fault, not something to impute.

Scoring, threshold, attribution

- **Scoring:** TranAD reconstructs only the **last timestep**; `score = 0.5.mean((x1-elem)2) + 0.5.mean((x2-elem)2)`. Squared error is taken *before* the feature-mean, yielding a **45-length per-feature error vector**.
- **Threshold:** `9.269035945180804e-05` (99th percentile of scores on the normal-validation split). Decision: `is_anomaly = score > threshold`.
- **Feature attribution:** the 45 per-feature errors → `top_features(n)` tells the operator **which sensors drove the alert**.

Conceptual "why" questions

- **Why 30 samples?** TranAD was trained with a 30-step context window (confirmed by `pos_encoder.pe = (30,1,90)`). It reconstructs the latest reading *given 30 steps of history*, so it needs exactly 30. At 5 s/sample that's 150 s of context.
- **Why 45 features?** SWaT has 51 process variables; 6 were near-constant ($\text{std} \leq 1\text{e-}8$) and dropped during research, leaving 45 informative sensors/actuators across the 6 treatment stages.
- **Why a rolling window?** Live telemetry arrives one sample at a time, but the model needs a full 30-sample matrix. `RollingWindow` accumulates samples and emits one fresh window per new sample (stride 1) once full.
- **What does the model output?** A scalar anomaly score for the window's last timestep, plus a 45-vector of per-feature reconstruction errors. The threshold turns the scalar into a boolean.
- **What does attribution tell us?** The sensors whose behaviour the model could least reconstruct — i.e. the likely locus of the abnormal process behaviour (process-aware evidence).

⚠ Model selection is complete. Do not suggest replacing TranAD. Artifacts are immutable. Do not retrain except in an explicitly-scoped future research experiment.

4. SWaT Dataset and Data Pipeline

Provenance

- **Source:** SWaT (Secure Water Treatment), iTrust / SUTD — a real 6-stage water treatment testbed. **Licensed; local-only; gitignored** (`.gitignore` excludes `*.csv`). Set `$SWAT_DATASET_DIR` (default `~/Downloads/SWaT/SWaT-dataset/`).
- **Files present locally:** `attack.csv` (54,621 rows, 100% Attack), `normal.csv` (1,387,098 rows, 100% Normal), `merged.csv` (1,441,719 rows — **excluded**).

What we use / exclude

- **We exclude** `merged.csv` — it carries the *identical* 991,800-row data gap as the raw concatenation, so it is not a cleaner source.
- **Attack_v0 (our replay source):** the reconstructed clean series = `attack.csv` (54,621) + `normal.csv` rows **1-395,298** (the Normal-labelled rows of the attack period), merged by Timestamp to restore the original **normal → attack → normal** ordering.

Reconstructed clean replay dataset — key numbers

Property	Value
Rows	449,919 (0 missing values, all 45 features populated)
True attack ratio	12.14%
Attack label segments	35 contiguous (cite 35 for labels; 36 for "attacks launched on the testbed")
Source rate	1 Hz → subsample 5 → 5 s effective period
Window span	$30 \times 5 \text{ s} =$ 150 s
Detection-latency floor	150 s — you cannot score a single reading; the buffer must fill first

Data-quality / provenance findings (and decisions)

1. **Duplicated normal-data issue** — `normal.csv` rows 395,299–1,387,098 (991,800 rows) are the same normal run present **twice** (Normal_v0 + Normal_v1, byte-identical on shared timestamps). 35.7% of normal rows are exact duplicates, which deflated the apparent attack ratio to 3.79%.
2. **Header mismatch** — 7 column names have a leading space in the attack file (`MV101` , etc.). When concatenated with the normal files, those 7 columns are **empty in all 991,800 block-2 rows**; the notebook's `.ffill().fillna(0)` then silently carried forward stale values.
3. **Frozen validation features** — because the validation split fell entirely inside block 2, **6 of 45 model features were frozen constants** (MV101, AIT201, MV201, P201, P204, MV303). The threshold was calibrated on this contaminated distribution → the nominal 1% FPR won't hold.
4. **P301 finding** — over the *clean* Attack_v0 slice, the only truly single-valued feature is **P301** (constant 1.0), **not** the scaler-frozen {P204, P206}. Reason: P204/P206 are constant only in *normal* operation but actuate under attack (the clean slice contains attacks). This is a data-characterization finding, not a reconstruction defect; a regression test now asserts the constant set is exactly `{P301}` .

What we decided: the raw defects belong to the research concat, not to our pipeline. Production preprocessing **rejects** non-finite input (never ffill). We replay the **clean Attack_v0** reconstruction. Artifacts and threshold are shipped **unchanged**, with the caveat documented rather than silently corrected.

5. Model Evaluation and Metrics

The shipped threshold

`threshold.json = 9.269035945180804e-05` , recorded as `percentile: 99` , `metric: tranad_score` , `window: 30` . It is the 99th percentile of TranAD scores on the **normal-validation** split. It is **not** the notebook's best-F1 (test-set-searched) threshold.

Why the calibration has a caveat

The validation split sat inside the duplicated block where 6/45 features were frozen. So the threshold's by-construction ~1% false-positive rate does **not** hold on complete telemetry — expect elevated FPR (measured: ~3%).

Why F1 = 0.9999 must NOT be the headline

That figure came from a **test-set-searched** threshold, which the notebook itself warns "can read ~1.0 even for a broken model." Presenting it would not survive examiner scrutiny. ⚠ Never present 0.9999 as model performance.

Point-wise vs point-adjusted

- **Point-adjusted:** if the model flags *any* point in an attack segment, the *whole* segment counts as detected. It only ever raises the score; Kim et al. (2022) show it systematically overstates performance. Conventional in SWaT literature but optimistic.
- **Point-wise:** each window judged on its own. Honest and deployment-relevant.

The honest "ladder" (from `docs/provenance/measured_detection_metrics.md`)

Rung	Dataset	Labelling	Protocol	F1
1	research concat (defective)	any-in-window	point-adjusted	0.7934 (reproduces published F1_fixed — validation control)
2	clean Attack_v0	any-in-window	point-adjusted	0.9012
3	clean Attack_v0	last-timestep	point-adjusted	0.8815
4	clean Attack_v0	last-timestep	point-wise	0.7822 ← DEPLOYMENT NUMBER

Deployment-oriented metrics (present these)

Metric	Value
Point-wise Precision	0.7822
Point-wise Recall	0.7821
Point-wise F1	0.7822 ← primary
False-positive rate	3.0117%
ROC-AUC (threshold-free)	0.9429
PR-AUC (threshold-free, true 12.15% base rate)	0.8516

How to present them in the MVP/defense

"At the shipped threshold, honest **point-wise F1 is 0.7822** (P 0.7822 / R 0.7821). We deliberately do **not** headline the point-adjusted 0.7934 or the test-searched 0.9999 — both overstate performance. Threshold-free discrimination is strong: **ROC-AUC 0.9429, PR-AUC 0.8516**. The measured **FPR is ~3%** vs a nominal 1%, which we *predicted* from a documented data defect and then confirmed. This honesty is a strength of the work."

Recalibration — discussed, ● DEFERRED

- **Why discussed:** the ~3% FPR comes from calibrating on a contaminated split; a clean-block recalibration could restore the ~1% target.
 - **Why deferred:** it requires explicit approval and would change reported behaviour; the team chose to ship-and-document for the MVP.
 - **What a future experiment means:** recalibrate the threshold on a clean block-1 validation subset, measure FPR/F1, compare.
 - **Hard rule:** it must **write a new sibling artifact**, never overwrite `threshold.json`, and must **not** retrain the model. ⚠ *Do not recalibrate now.*
-

6. Phase-by-Phase History

⚠ **The actual build order differs from the original roadmap.** The roadmap listed backend (Phase 2) before telemetry replay (Phase 3). In reality **Phase 2A (replay + rolling window) was built before Phase 2B (FastAPI backend)** — deliberately, so the backend could be designed as a thin, stateless API with buffering owned entirely by the caller.

Phase	Objective	What was implemented	Major files	Key decisions	Tests	Issues discovered	Status	Commit	Deferred
0	Foundation / provenance	Reverse-engineered & documented the artifact + preprocessing contract; pinned environment	<code>docs/provenance/tranad_swat_provenance.md</code> , <code>ml/configs/tranad_swat.yaml</code>	Artifacts immutable; env pins (sklearn 1.7.2, numpy≥2, torch 2.9.1)	—	Export script lost; notebooks had zero saved outputs	● COMPLETE	<code>cdabb68</code> (+ <code>5e3fbec</code> , <code>9b31fc9</code>)	—
1	Production inference library	<code>TranADDetector</code> facade + preprocessing/model/scoring; golden vectors	<code>ml/src/{model,preprocessing,scoring,detector}.py</code> , <code>tests/fixtures/golden_vectors.json</code>	Reject non-finite input; expose 45 feature errors; detector owns no buffer	~95	Silent-failure risks (flatten/clip/order)	● COMPLETE	<code>43297ac</code>	threshold recalibration
1.5	Honest evaluation	Clean Attack_v0 loader, metrics, measured detection report	<code>ml/src/{dataset,metrics}.py</code> , <code>scripts/measure_detection_metrics.py</code> , <code>docs/provenance/measured_detection_metrics.md</code>	Report point-wise F1 0.7822, not 0.9999	+metrics/dataset tests	Data defects (dup block, frozen features, P301)	● COMPLETE	<code>64c3b63</code>	recalibration
2A	Replay + buffering (before 2B)	<code>SWatReplay</code> , <code>RollingWindow</code> (owns 30-sample buffer)	<code>simulator/swat_replay.py</code> , <code>simulator/window_buffer.py</code>	Buffer lives in <code>RollingWindow</code> , not detector/API	replay + buffer tests	—	● COMPLETE	<code>6d7a50f</code>	—
2B	FastAPI backend	<code>GET /health</code> , <code>POST /score</code> ; thin stateless boundary; two distinct 422 shapes	<code>backend/app.py</code> , <code>backend/schemas.py</code>	API keeps no streaming state; enforce feature order; honest 503 health	22 (suite=238)	—	● COMPLETE	<code>772210b</code>	—
3	End-to-end integration	<code>pipeline.py</code> (injected scorer), demo script; verified on real SWaT over in-process + real socket	<code>simulator/pipeline.py</code> , <code>scripts/run_e2e_demo.py</code>	Inject transport; simulator imports no torch/fastapi/ml.src	12	—	● COMPLETE	<code>dfb7bf8</code>	alert engine, dashboard
4	Alert engine (stateful)	Dedup + OPEN→CLOSE lifecycle, heuristic severity, verbatim attribution	<code>alerting/engine.py</code> , <code>alerting/alert.py</code>	Severity is a heuristic, not ML; in-memory only; duck-typed	21	—	● COMPLETE	<code>6c3ee95</code>	persistence
5A	Monitoring API + dashboard	<code>/status</code> , <code>/alerts</code> , <code>/alerts/active</code> ; zero-build polling dashboard; dev CORS	<code>backend/app.py</code> (+), <code>frontend/index.html</code>	<code>/score</code> contract unchanged; explicit CORS; alerts in-memory	15 (suite=286)	—	● COMPLETE (current HEAD)	<code>fc79a4a</code>	database, auth, notifications

7. Current MVP

The MVP is a **working local end-to-end detection + alerting + monitoring system**, verified on real SWaT data over both an in-process ASGI app and a real uvicorn HTTP socket.

Capabilities (all ● IMPLEMENTED)

- **Normal scenario:** replay a clean segment → windows score below threshold → dashboard shows NORMAL, no active alert.
- **Attack scenario:** replay an attack segment → scores exceed threshold → anomalies flagged (100% detection on the demo's auto-selected clearest segment).

- **Alert creation:** first anomalous window opens an `Alert` (deterministic id).
- **Alert deduplication:** consecutive anomalous windows extend the *same* alert (window_count and span grow; peak score/attribution/severity tracked) — 50 anomalous windows = 1 incident.
- **Alert lifecycle & recovery:** first normal window after an anomaly **closes** the alert (sets `closed_at`).
- **Dashboard:** system status, live detection status + score sparkline, active alert (id, severity, peak score, top features, window span/count), alert history.
- **Monitoring API:** `/status`, `/alerts?limit=&status=`, `/alerts/active`.
- **Real HTTP operation:** demo can run against a live `uvicorn backend.app:app`.
- **Real SWaT replay:** driven from the reconstructed clean Attack_v0.

End-to-end flow (as coded)

```
normal telemetry -> RollingWindow -> /score -> TranAD -> score <= thr -> is_anomaly=false -> (no alert) -> dashboard NORMAL
attack telemetry -> RollingWindow -> /score -> TranAD -> score > thr -> is_anomaly=true -> AlertEngine -> OPEN alert (severity, top features) -> dashboard ANOMALY + active alert
return to normal -> RollingWindow -> /score -> TranAD -> score <= thr -> is_anomaly=false -> AlertEngine -> CLOSE alert -> dashboard: alert moves to history
```

Known MVP limitations

- Alert state is **in-memory** — lost on restart.
- **150 s** detection-latency floor (buffer must fill).
- FPR ~3% (documented caveat).
- No persistence, auth, notifications, containerization, or cloud yet.

8. MVP Smoke Test / Manual Demo

Verified manual sequence (from `scripts/run_e2e_demo.py`, `backend/README.md`, `frontend/README.md`)

1. **Backend startup:** `uvicorn backend.app:app` → `http://127.0.0.1:8000` (detector lazy-loads on first request; `/docs` available).
2. **Frontend startup:** `cd frontend && python -m http.server 5500` → open `http://127.0.0.1:5500/` (CORS allow-lists `:5500`).
3. **Endpoints tested:** `/health` (200 + diagnostics), `/score`, `/status`, `/alerts`, `/alerts/active`.
4. **Normal mode:** `python scripts/run_e2e_demo.py --mode normal` → prints per-window score/threshold, ends with false-positive count and the FPR caveat note.
5. **Attack mode:** `python scripts/run_e2e_demo.py --mode attack` → auto-selects the clearest attack segment, prints anomalies + top contributing features, ends with recall.
6. **Recovery:** naturally shown because attack segments are embedded chronologically in normal telemetry; the alert closes on the first normal window after the attack.
7. **Real HTTP verification:** add `--base-url http://127.0.0.1:8000` to drive a live socket instead of in-process ASGI.
8. **Dashboard behaviour:** polls every 2 s; shows status, detection, active alert, history; graceful red banner if backend unavailable; explicit empty states.

The two known demo observations

1. **Timestamp ordering when replaying disjoint segments.** If you replay non-adjacent segments back-to-back, the echoed `window_end` timestamps can appear non-monotonic where segments join. → **Sequence/cosmetic behaviour, not a functional defect.** The detector never uses timestamps; they are echoed metadata. *Workaround for the demo:* replay a single contiguous range (the demo's default `normal_range` / single `attack_range`), so timestamps stay ordered.
2. **Auto-select attack probing can mutate alert state.** `--mode attack` auto-selection *probes* each candidate segment's first few windows by actually calling the scorer. If that scorer points at the **live monitoring backend**, those probe windows feed the AlertEngine and can create/adjust alerts before the "real" run. → **Demo limitation, not a functional defect.** *Workaround:* either pass an explicit `--attack-segment N` (skips probing), or run auto-select against a fresh/in-process app and only point the *final* run at the live backend, or restart the backend between probe and demo

(alerts are in-memory).

Both are demo-orchestration artifacts; neither indicates a bug in detection, buffering, or the alert lifecycle.

9. What Is Left After the MVP

Currently only empty scaffold directories exist for infra: `docker/`, `infrastructure/terraform/`, `infrastructure/kubernetes/`, `.github/workflows/` are **empty** (nothing tracked). So Docker/Terraform/K8s/CI are ● PLANNED, not implemented.

A. Must-have before final submission

Item	Status	Why
Alert persistence / database	● DEFERRED → must build	Alerts vanish on restart; final demo needs durable incident history
Docker (backend + dashboard images)	● empty <code>docker/</code>	Reproducible deployment; prerequisite for cloud/K8s
CI pipeline (<code>.github/workflows/</code> empty)	●	Run the 286 tests automatically on push; evidence of quality
Basic DevSecOps security gates (SAST, dependency scan, container scan)	●	The proposal is explicitly a <i>DevSecOps</i> project — this is core, not optional
Stuck-channel detection in ingest	● (provenance open-item #4)	Non-finite is rejected, but a channel frozen at a <i>plausible</i> constant still passes silently
End-to-end / deployment reproducibility docs	partial ● (READMEs)	Examiner must be able to reproduce

B. High-value improvements

Item	Status	Why
AWS deployment	●	Proposal calls for cloud; strong demo value
Terraform (IaC)	● empty dir	Reproducible, reviewable infra
Authentication on API/dashboard	●	Operator tooling should not be open
Notifications (email/Slack/webhook on alert)	●	Completes the alerting story
Observability/logging (structured logs, metrics)	partial ● (Python logging exists)	Production readiness
Threshold recalibration experiment (sibling artifact)	● DEFERRED	Could restore ~1% FPR; needs approval, never overwrites
Performance/latency testing	○	Quantify throughput; validate the 150 s floor claim

C. Optional stretch goals

Item	Status
Kubernetes (empty <code>infrastructure/kubernetes/</code>)	● — only if time allows after Docker+cloud
Reliability/chaos testing	○
Advanced dashboard (WebSocket/SSE instead of polling, charts)	○
Multi-model fusion (research only — do NOT touch the MVP detector)	○

10. Post-MVP Roadmap

CURRENT MVP (Phase 5A) [COMPLETE]

6

Hardening & stuck-channel detection

7

Alert persistence (database)

8

Containerization (Docker)

9

CI/CD pipeline

10

DevSecOps security gates (SAST + deps + container scan)

11

Cloud deployment (AWS) + Terraform

12

Kubernetes (stretch)

13

Final testing, hardening, observability

14

Final documentation + demo

Phase	Objective	Deliverables	Why it matters	Depends on	Complexity	Demonstrate
6 Hardening	Close ingest gaps	Stuck-channel check; input hardening; logging polish	Detection integrity	MVP	Low-Med	Frozen-but-plausible channel is caught
7 Persistence	Durable alerts	DB (SQLite/Postgres) behind AlertEngine; alerts survive restart; history API reads from store	Final demo credibility	6	Med	Restart backend, alerts remain
8 Docker	Reproducible images	Dockerfile(s) for backend + dashboard; compose for local stack	Prereq for cloud/K8s/CI	7	Med	<code>docker compose up</code> runs the whole stack
9 CI/CD	Automated quality	GitHub Actions: run 286 tests + lint on push; build images	Evidence of engineering rigor	8	Med	Green pipeline on a PR
10 Security gates	DevSecOps core	SAST (e.g. Bandit/CodeQL), dependency scan, container/image scan wired into CI; fail-on-severity gate	This is a DevSecOps graduation project	9	Med-High	A scan blocking a bad build
11 AWS + Terraform	Cloud deploy	IaC provisioning; deployed backend + dashboard reachable	Proposal requirement; big demo value	8,10	High	Live cloud URL detecting an attack
12 Kubernetes	Orchestration (stretch)	K8s manifests; horizontal scaling of stateless detector	Shows scalability of the stateless design	11	High	Scale detector replicas
13 Final testing/observability	Production readiness	Perf/latency tests, structured logs/metrics dashboards, security hardening	Confidence + evidence	11	Med	Metrics + load results
14 Docs + demo	Delivery	Architecture docs, runbook, final attack demo script	Grading	all	Low-Med	Full narrated end-to-end

11. Team Work Distribution

The clean layer boundaries (simulator $\perp$ backend $\perp$ ml.src $\perp$ alerting) make parallel work safe **as long as shared interfaces have single owners**.

Area	Scope	Owns	Shared interface (single-owner)
ML / detection	Preserve TranAD contract; run recalibration <i>experiment</i> if approved (sibling artifact only); metrics	ml/src , ml/artifacts (read-only), docs/provenance	The preprocessing/scoring contract — one owner , no one else edits ml/src
Backend / API	FastAPI endpoints, validation, health/status, auth	backend/	The /score + monitoring API schema (backend/schemas.py) — one owner
Alerting / persistence	AlertEngine + database backing; notifications	alerting/ , new persistence module	The Alert DTO + AlertEngine.process() signature — one owner
Frontend / dashboard	Dashboard, UX, charts	frontend/	Consumes API only; depends on backend schema owner
Infrastructure / cloud	Docker, Terraform, AWS, K8s	docker/ , infrastructure/	Container/deploy contract (ports, env, health probe) — coordinate with backend owner
DevSecOps / CI-CD	CI pipeline, SAST, dependency + container scanning, gates	.github/workflows/	Test-run + build commands — coordinate with backend + infra
Testing / documentation	Grow E2E tests, perf tests, architecture docs, runbook, demo scripts	tests/ , docs/ , scripts/	Owns demo orchestration (avoids the Part 8 probing pitfall)

Interfaces that MUST be single-owned to avoid conflicts: (1) the ML preprocessing/scoring contract, (2) the API request/response schema, (3) the Alert DTO + engine signature, (4) the container/deploy contract. Everything else can proceed in parallel.

Do NOT parallelize onto the model itself — ml/artifacts is immutable; only the ML owner runs the (approved, sibling-only) recalibration experiment.

12. MVP Presentation Plan

Live-demo sequence

#	Show	Say	Proves
1	Start uvicorn + dashboard	"Backend is a thin stateless API; dashboard is separate and polls it."	Clean separation
2	/health = 200, dashboard status OK	"Detector loaded; model = TranAD; threshold shown honestly."	Healthy readiness
3	Normal replay running	"45 process variables, 30-sample windows = 150 s context."	Real telemetry pipeline
4	Detection status NORMAL, score < threshold	"Every window scored; below threshold = normal."	Live detection
5	Trigger attack replay	"Now we replay a real SWaT attack segment."	Real attack data
6	Anomaly score jumps above threshold	"Reconstruction error spikes — TranAD can't explain the state."	Model detects attack
7	Active alert OPENS on dashboard	"Consecutive anomalies become ONE deduplicated incident."	Alert lifecycle
8	Top contributing features	"These sensors drove the alert — process-aware evidence."	Attribution / explainability
9	Alert history list	"Every incident is recorded with peak score and span."	Monitoring
10	Return to normal	"Telemetry returns to normal."	Recovery
11	Alert CLOSEs → moves to history	"Incident closes automatically."	Full lifecycle
12	State limitations slide	"Alerts are in-memory; FPR ~3% by a documented data caveat; next: persistence + cloud."	Honesty + roadmap

Likely examiner questions & concise answers

- **Why TranAD?** Selected after a research comparison of multiple models; strong threshold-free discrimination (ROC-AUC 0.94) and good reconstruction-based detection on multivariable process data. Selection is closed.
- **Why SWaT?** A real, labelled water-treatment testbed dataset — the closest public analog to our chlorination use case.
- **Why 30 samples?** The model was trained with a 30-step context (encoded in its positional encoding); at 5 s/sample that's 150 s of history.
- **Why 45 features?** SWaT's 51 process variables minus 6 near-constant columns dropped in research.
- **What does F1 = 0.7822 mean?** Honest point-wise F1 at the shipped threshold — balanced ~78% precision and recall per window.
- **Why is FPR 3%?** The threshold was calibrated on a split contaminated by a documented SWaT data defect (6 frozen features); we predicted and confirmed the elevated rate rather than hiding it.
- **Why not use F1 = 0.9999?** It came from a test-set-searched threshold that inflates to ~1 even for broken models — not a valid deployment figure.
- **What does severity mean? Is it ML?** No — it's a transparent engineering heuristic (score/threshold ratio banded LOW/MEDIUM/HIGH), explicitly not a model output.
- **Why is alert state in memory?** MVP scope; persistence is the next planned phase.
- **What if the backend restarts?** Alerts are lost today — that's why persistence is priority-ranked next.
- **How scalable is it?** The detector and API are stateless, so they scale horizontally; only the AlertEngine (and future DB) holds state.
- **Why is the detector stateless / buffering outside FastAPI?** So the model layer stays trivially testable and horizontally scalable; `RollingWindow` is the single owner of the buffer.

13. Final Project Presentation Plan

The final demo should extend the MVP demo with production/DevSecOps evidence (include only what is actually completed by then):

Final-demo element	Beyond MVP	Realistic?
Complete detection pipeline	Same, but deployed	✓
Persistent alerts	Restart backend → history survives	✓ (Phase 7)
Production-style deployment	Dockerized stack	✓ (Phase 8)
Secure CI/CD	Green pipeline, tests auto-run	✓ (Phase 9)
Security scanning	SAST + dependency + container scan gates	✓ (Phase 10) — core to the DevSecOps thesis
Cloud deployment	Live AWS URL detecting an attack	✓ (Phase 11)
Infrastructure as code	<code>terraform apply</code> provisioning	✓ (Phase 11)
Kubernetes	Scale stateless detector replicas	⚠ stretch — only if completed
Observability	Logs/metrics dashboard	✓ (Phase 13)
Security hardening	Auth, least-privilege, no secret leakage	✓
Attack demonstration	Same narrative, on cloud	✓
Test evidence	286+ tests green in CI, perf results	✓

Narrative shift: MVP proves "the detection works"; the final proves "it's a deployable, secured, reproducible DevSecOps product."

14. Project Status Table

Area	Current status	Completion	Next action
ML (TranAD)	Selected, integrated, immutable artifacts	● 100% (frozen)	Do not change; optional sibling recalibration (approval only)
Dataset	Clean Attack_v0 reconstructed, provenance documented	● 100%	None
Preprocessing	Production contract, non-finite rejected, tested	● 100%	None
Inference	TranADDetector, stateless, golden-vector tested	● 100%	None
Replay	SWaTReplay, segment API	● 100%	None
Windowing	RollingWindow owns 30-sample buffer	● 100%	None
FastAPI	/health, /score, thin/stateless	● 100%	Add auth later
Alerting	Dedup + lifecycle + severity heuristic	● 100% (in-memory)	Add persistence
Monitoring API	/status, /alerts, /alerts/active	● 100%	None
Dashboard	Vanilla polling operator view	● 100%	Enhance after core
E2E	Verified in-process + real HTTP	● 100%	Grow scenarios
Persistence	—	● 0% (in-memory)	Build database backing
Docker	Empty docker/ dir	● 0%	Write Dockerfiles + compose
CI/CD	Empty .github/workflows/	● 0%	Add test/build pipeline
Security (DevSecOps)	—	● 0%	SAST + deps + container scan gates
AWS	—	● 0%	Deploy after Docker
Terraform	Empty infrastructure/terraform/	● 0%	IaC after Docker
Kubernetes	Empty infrastructure/kubernetes/	● 0%	Stretch, after cloud
Testing	286 tests passing	● ~85% (unit/integration strong; no perf/deploy)	Add perf + deploy tests
Documentation	Provenance + READMEs strong	● ~60%	Architecture doc + runbook
Demo	Working local demo + observations	● ~80%	Harden demo orchestration (Part 8 workarounds)

15. Executive Summary for the Team

Where are we now? A complete, tested, honest **local MVP** (Phase 5A). Real SWaT telemetry flows through replay → rolling window → stateless TranAD detector → in-memory alert engine → monitoring API → operator dashboard, verified over real HTTP. 286 tests pass; the working tree is clean.

What can we show in the MVP? A live normal→attack→recovery demo: normal telemetry reads normal, a real SWaT attack spikes the anomaly score above threshold, one deduplicated alert opens with severity

- the exact contributing sensors, the dashboard reflects it, and the alert closes on recovery. Plus honest metrics (point-wise F1 0.7822, ROC-AUC 0.94) with a transparent FPR caveat.

What remains? Persistence (alerts are in-memory), containerization, CI/CD, DevSecOps security gates, and cloud deployment (AWS/Terraform, optionally K8s). The infra directories exist but are **empty scaffolds**.

What should we improve after the MVP? In priority order: durable alert storage → Docker → CI with security scanning → cloud deployment → observability/hardening. Also close the stuck-channel ingest gap and (with approval) run the sibling-artifact threshold recalibration experiment.

What must we NOT change? ⚠ Do not replace or retrain TranAD; artifacts are immutable. Do not move buffering into FastAPI or make the detector stateful. Do not overwrite `threshold.json`. Do not present point-adjusted or 0.9999 F1 as the deployment number. Keep severity labelled as a heuristic, not ML.

Recommended path to final delivery: Hardening → Persistence → Docker → CI/CD → Security gates → AWS + Terraform → (K8s stretch) → final testing/observability → docs + demo.

Key immutable decisions (must be respected by everyone)

- TranAD is the actual MVP model; model selection is complete.
- The five artifacts under `ml/artifacts/swat_TranAD/` are immutable / byte-identical.
- `RollingWindow` owns the 30-sample buffer.
- `TranADDetector` is stateless.
- FastAPI is a thin boundary; it does not buffer or hold streaming state.
- Threshold recalibration is deferred; never overwrite `threshold.json`.
- Severity is an engineering heuristic, not an ML output.
- Alert state is currently in-memory.
- Clean Attack_v0 is the replay source; `merged.csv` is excluded.
- Point-wise F1 = 0.7822 is the deployment metric; never headline point-adjusted 0.7934 or 0.9999.

16. Next 3 Priorities

Priority 1 — Alert persistence (database backing the AlertEngine). Highest demo credibility for the smallest architectural change. Add a datastore behind the existing `AlertEngine` / `Alert` DTO so incidents survive a restart and the history API reads from it. Owner: alerting/persistence. Keep the engine's public interface unchanged so backend/frontend need no changes. *(Directly removes the biggest stated MVP limitation.)*

Priority 2 — Containerization + CI with security gates (Docker → GitHub Actions → SAST/dependency/container scan). This is the core of the *DevSecOps* thesis and is currently 0% (empty scaffolds). Dockerize backend + dashboard, wire a pipeline that runs the 286 tests and builds images, then add SAST + dependency + container scanning as failing gates. Owner: infra + DevSecOps, coordinating the container/deploy contract with the backend owner. *(Prerequisite for cloud and the strongest "engineering rigor" evidence.)*

Priority 3 — Cloud deployment with Terraform (AWS). Highest final-demo value: a live cloud URL detecting a replayed SWaT attack, provisioned reproducibly via IaC. Depends on Priorities 1 and 2. Owner: infrastructure/cloud. *(Fulfills the original proposal's cloud requirement and headlines the final presentation.)*

End of briefing. Generated read-only from the repository; no source files were modified.